

```

// -----
// KEYBOARD-PINS (manuell scanning)
// -----
const byte KEYPAD_PINS[7] = {2, 3, 4, 5, 6, 7, 8};

// -----
// MENY-STATES
// -----
enum State {
    MAIN_MENU,
    INPUT_VALUE
};

State currentState = MAIN_MENU;

// -----
// PARAMETRE
// -----
enum Param {
    PARAM_NONE,
    PARAM_NUMPIX,
    PARAM_R,
    PARAM_G,
    PARAM_B
};

Param currentParam = PARAM_NONE;

// -----
// NUMERISK INPUT-STRUKTUR (matcher powerpoint)
// -----
struct NumericInput {
    bool active = false;
    bool cancelled = false;
    String buffer = "";
    int result = 0;
};

NumericInput numIn;

// -----
// SETUP
// -----
void setup() {
    Serial.begin(9600);

    for (int i = 0; i < 7; i++) {

```

```

    pinMode(KEYPAD_PINS[i], INPUT_PULLUP);
}

Serial.println("System klar.");
showMainMenu();
}

// -----
// LOOP
// -----
void loop() {

    char key = getKeyPressed();

    // håndter tall-input
    if (numIn.active) {
        if (updateNumericInput(key)) {
            if (!numIn.cancelled) {
                applyParameter(currentParam, numIn.result);
            }
            showMainMenu();
        }
        return;
    }

    // håndter hovedmeny
    if (key != 0) {
        handleMainMenu(key);
    }
}

// -----
// VIS MENY
// -----
void showMainMenu() {
    currentState = MAIN_MENU;
    Serial.println("\n--- HOVEDMENY ---");
    Serial.println("1 = Farge (sett R)");
    Serial.println("2 = Lysstyrke (dummy)");
    Serial.println("3 = Modus");
    Serial.println("* = Sett G");
    Serial.println("# = Sett B");
}

// -----

```

```
// HÅNDTER MENYVALG
// -----
void handleMainMenu(char key) {
  switch (key) {
    case '1':
      startInputMode(PARAM_R, "R-verdi (0–255)");
      break;

    case '2':
      Serial.println("Lysstyrke ikke implementert ennå.");
      break;

    case '3':
      Serial.println("Modus ikke implementert ennå.");
      break;

    case '*':
      startInputMode(PARAM_G, "G-verdi (0–255)");
      break;

    case '#':
      startInputMode(PARAM_B, "B-verdi (0–255)");
      break;

    default:
      Serial.println("Ugyldig tast.");
  }
}
```

```
// -----
// START NUMERISK INPUT
// -----
void startInputMode(Param p, const char* message) {
  currentParam = p;
  startNumericInput();
  Serial.println(message);
}
```

```
// -----
// START INPUT
// -----
void startNumericInput() {
  numIn.active = true;
  numIn.cancelled = false;
  numIn.buffer = "";
  Serial.println("Skriv tall. * = OK, # = Avbryt");
}
```

```
}
```

```
// -----
```

```
// OPPDATER INPUT
```

```
// -----
```

```
bool updateNumericInput(char key) {  
    if (!numIn.active || key == 0) return false;
```

```
    if (key == '#') {  
        numIn.cancelled = true;  
        numIn.active = false;  
        Serial.println("Avbrutt.");  
        return true;  
    }
```

```
    if (key == '*') {  
        numIn.result = numIn.buffer.toInt();  
        numIn.active = false;  
        Serial.print("Verdi bekreftet: ");  
        Serial.println(numIn.result);  
        return true;  
    }
```

```
    if (key >= '0' && key <= '9') {  
        numIn.buffer += key;  
        Serial.print("> ");  
        Serial.println(numIn.buffer);  
    }
```

```
    return false;  
}
```

```
// -----
```

```
// PARAMETER-APPLY
```

```
// -----
```

```
int R = 0, G = 0, B = 0;
```

```
void applyParameter(Param p, int value) {  
    value = constrain(value, 0, 255);
```

```
    switch (p) {  
        case PARAM_R:  
            R = value;  
            Serial.print("R satt til: "); Serial.println(R);  
            break;  
        case PARAM_G:
```

```

    G = value;
    Serial.print("G satt til: "); Serial.println(G);
    break;
case PARAM_B:
    B = value;
    Serial.print("B satt til: "); Serial.println(B);
    break;
default:
    break;
}

Serial.print("Ny farge: R=");
Serial.print(R); Serial.print(" G=");
Serial.print(G); Serial.print(" B=");
Serial.println(B);

// her kobler du senere til NeoPixel.update()
// updateNeopixelColor();
}

// -----
// MANUELL KEYPAD-SKANNING
// -----
char getKeyPressed() {
    bool anyLow = false;
    for (int i = 0; i < 7; i++) {
        if (digitalRead(KEYPAD_PINS[i]) == LOW) {
            anyLow = true;
            break;
        }
    }
    if (!anyLow) return 0;

    for (int i = 0; i < 7; i++) {
        pinMode(KEYPAD_PINS[i], OUTPUT);
        digitalWrite(KEYPAD_PINS[i], LOW);

        for (int j = 0; j < 7; j++) {
            if (i != j) {
                pinMode(KEYPAD_PINS[j], INPUT_PULLUP);
                if (digitalRead(KEYPAD_PINS[j]) == LOW) {
                    delay(50);
                    while (digitalRead(KEYPAD_PINS[j]) == LOW);
                    return mapKey(i + 2, j + 2);
                }
            }
        }
    }
}

```

```
    pinMode(KEYPAD_PINS[i], INPUT_PULLUP);
}
return 0;
}
```

```
// -----
// MAPKEY — UENDRET FRA TIDLIGERE VERSJON
// -----
char mapKey(int a, int b) {
    int minP = min(a, b);
    int maxP = max(a, b);
    if (minP == 3 && maxP == 4) return '1';
    if (minP == 2 && maxP == 3) return '2';
    if (minP == 3 && maxP == 6) return '3';
    if (minP == 4 && maxP == 8) return '4';
    if (minP == 2 && maxP == 8) return '5';
    if (minP == 6 && maxP == 8) return '6';
    if (minP == 4 && maxP == 7) return '7';
    if (minP == 2 && maxP == 7) return '8';
    if (minP == 6 && maxP == 7) return '9';
    if (minP == 2 && maxP == 5) return '0';
    if (minP == 4 && maxP == 5) return '*';
    if (minP == 5 && maxP == 6) return '#';
    return '?';
}
```